

EABench: A Configurable Benchmark Framework for Evaluating LLM-Powered Enterprise Agents

EABench Contributors
<https://github.com/xhyccc/EABench>

Abstract

Large language model (LLM) agents are increasingly deployed in enterprise productivity settings—automatically managing emails, scheduling meetings, retrieving files, and executing multi-step workflows across heterogeneous data sources. Despite rapid progress in general-purpose agent benchmarks, existing evaluation frameworks fall short in two important respects: they provide little support for generating *realistic enterprise data at scale*, and they offer limited flexibility in *agent architecture configuration*. We present **EABench**, an open-source benchmark framework designed to address both gaps. EABench provides (1) an LLM-driven *synthetic data generation* pipeline that produces coherent, inter-connected corporate corpora spanning emails, meetings, files, chats, and channels; (2) a *configurable agent runtime* that supports multiple execution strategies (ReAct and Plan-and-Execute) and multiple LLM backends through a provider-agnostic interface; (3) a comprehensive *enterprise tool suite* encompassing semantic search over all content types, sandboxed file I/O, and code execution; and (4) a *multi-dimensional evaluation harness* combining deterministic assertions, citation relevance scoring, and LLM-as-a-Judge for end-to-end and user-level evaluation. Together, these components enable systematic, reproducible comparison of agent designs under realistic enterprise conditions.

1 Introduction

The emergence of LLM-based autonomous agents capable of using tools, planning multi-step workflows, and interacting with external APIs has prompted a wave of benchmarks aimed at measuring agent capability. However, a growing gap exists between academic benchmark environments and the conditions found in real enterprise deployments. Enterprise AI agents must navigate large, *heterogeneous* corpora—email threads, calendar invites, document repositories, instant messages, and project channels—where information is *fragmented* across sources, *access-controlled* at the user level, and *temporally distributed* across weeks or months of organizational history.

Current benchmarks such as WebArena [Zhou et al., 2023], AgentBench [Liu et al., 2023], and ToolBench [Qin et al., 2023] focus primarily on web navigation, software engineering tasks, and API calling, respectively. While these settings surface important capabilities, they do not

capture the *enterprise information retrieval and synthesis* tasks that constitute the bulk of knowledge-worker workload: “Summarize all action items from last week’s incident post-mortems,” “Find the latest contract negotiation emails with Vendor X,” or “Who owns the on-call rotation this month?”

EABench directly targets this gap. Its three core contributions are:

1. **Realistic, configurable data generation.** EABench uses an LLM-driven pipeline to generate fully synthetic yet coherent enterprise tenants at any scale, with user-defined industry, company size, timeline, and storyline events. The resulting corpora are inter-connected: emails reference meeting decisions, files document project outcomes, and chats discuss ongoing incidents—exactly as in real organizations.
2. **Flexible agent architecture.** Rather than fixing a single agent design, EABench exposes a *configuration-as-code* model in which every aspect of an agent—LLM backend, execution strategy (ReAct vs. Plan-and-Execute), available tools, system prompt, and planning prompt—is specified in a human-readable YAML file. This enables controlled ablation studies and rapid iteration without modifying source code.
3. **Comprehensive evaluation.** EABench’s evaluation harness combines *deterministic assertions* (verifiable ground-truth checks), *citation scoring* (verifying that responses cite real, relevant source entities), and *LLM-as-a-Judge* scoring for qualitative dimensions such as faithfulness, completeness, and reasoning quality. User-level evaluation simulates realistic access-control scenarios by running queries under specific user identities that determine data visibility.

The remainder of this paper is organized as follows. Section 2 surveys related work. Section 3 describes EABench’s methodology in detail. Section 4 presents experimental results and analysis. Section 5 concludes with a discussion of limitations and future directions.

2 Related Work

2.1 Agent Benchmarks

WebArena [Zhou et al., 2023] provides a realistic web browsing environment with four web applications

(shopping, reddit, GitLab, map) and 812 long-horizon tasks. While WebArena evaluates task completion in a realistic setting, its tasks are scoped to web interfaces and do not address enterprise knowledge retrieval. **WorkArena** [Drouin et al., 2024] extends this approach to ServiceNow workflows, targeting enterprise service management tasks. However, its data is limited to a single application and does not cover the richly inter-connected multi-source data of a real enterprise.

AgentBench [Liu et al., 2023] is a multi-dimensional benchmark spanning eight environments including OS, database, web browsing, and lateral reasoning tasks. It demonstrates that current LLMs struggle with multi-step tool-use tasks; however, its environments are largely synthetic or game-like rather than representative of knowledge-worker contexts.

ToolBench [Qin et al., 2023] focuses specifically on evaluating LLMs’ ability to invoke real-world APIs from a large catalog of 16,000 APIs. While this directly tests tool-use, the tasks are API-centric rather than information-synthesis-centric, and there is no notion of user-level access control or data provenance.

τ -**bench** [Yao et al., 2024] introduces the concept of user simulation for interactive task evaluation in airline and retail domains. The approach of simulating realistic user interactions—rather than specifying fully deterministic tasks—is conceptually related to EABench’s user-level evaluation mode, though τ -bench targets customer service scenarios rather than enterprise knowledge work.

2.2 Enterprise AI Assistants

Several industry systems have been deployed for enterprise AI assistance. **Microsoft 365 Copilot** [Microsoft, 2023] integrates LLMs with Microsoft Graph to answer queries over emails, documents, meetings, and Teams chats. Its architecture involves retrieval-augmented generation (RAG) over enterprise data with user-level access control. While such systems demonstrate the commercial viability of enterprise agents, they do not provide open evaluation datasets or benchmarking infrastructure.

Glean and similar enterprise search platforms combine vector search with access control lists (ACLs) to implement personalized search across connected SaaS applications. These systems illustrate the importance of user-context-aware retrieval—a feature EABench explicitly supports through its user-level indexing and filtering.

2.3 Synthetic Data Generation for Benchmarks

Several recent works have investigated LLM-generated synthetic data for evaluation. **AgentInstruct** [Mitra et al., 2024] uses LLMs to generate instruction tuning data by transforming raw documents through a pipeline of multiple specialized agents. **FrontierMath** [Glazer et al., 2024] demonstrates that LLM-generated problems can challenge state-of-the-art models. **WildChat** [Zhao et al., 2024] collects naturally occurring LLM conversations in the wild.

EABench’s approach differs from prior work in that it generates *structured organizational corpora* rather than isolated question-answer pairs. The generator produces inter-connected narrative artifacts (emails, meetings, files, chats) grounded in a shared storyline, enabling queries that require multi-hop reasoning across document types and temporal reasoning over event sequences.

2.4 LLM-as-a-Judge Evaluation

Automated evaluation using LLMs as judges has gained significant traction following MT-Bench [Zheng et al., 2023] and Chatbot Arena [Chiang et al., 2024]. LLM judges show high correlation with human preferences at lower cost and enable evaluation of open-ended responses that resist automatic metrics.

EABench extends the LLM-as-a-Judge paradigm to *agentic evaluation*, where the judge must assess not only the final response but also the *reasoning process*—including whether cited sources are accurate, whether tool calls were appropriate, and whether the overall research plan was followed.

2.5 ReAct and Plan-and-Execute Agent Frameworks

ReAct [Yao et al., 2022] introduced the Reason+Act paradigm, where LLMs interleave reasoning traces with tool invocations. This approach demonstrated that explicitly reasoning about the next action before taking it substantially improves task performance on knowledge-intensive tasks.

Plan-and-Solve [Wang et al., 2023] agents extend this idea by separating the planning phase—generating a high-level task decomposition—from the execution phase. This separation can improve reliability on complex multi-step tasks by preventing the agent from “losing the thread” during long tool-use sequences.

EABench implements both paradigms as first-class, YAML-configurable execution strategies, enabling direct comparison of their performance characteristics across benchmark tasks.

3 Methodology

EABench is organized around four interacting subsystems: (1) the data generation pipeline, (2) the configurable agent runtime, (3) the enterprise tool suite, and (4) the evaluation harness. Figure 1 shows the overall system architecture.

3.1 Data Generation

Design goals. Benchmark data for enterprise agents must satisfy three requirements simultaneously: (a) *realism*—the content must resemble actual corporate communication artifacts; (b) *coherence*—distinct artifacts (emails, meetings, files) must form a consistent narrative rather than isolated documents; and (c) *controllability*—the researcher must be able to specify the domain, scale, and scenario characteristics.

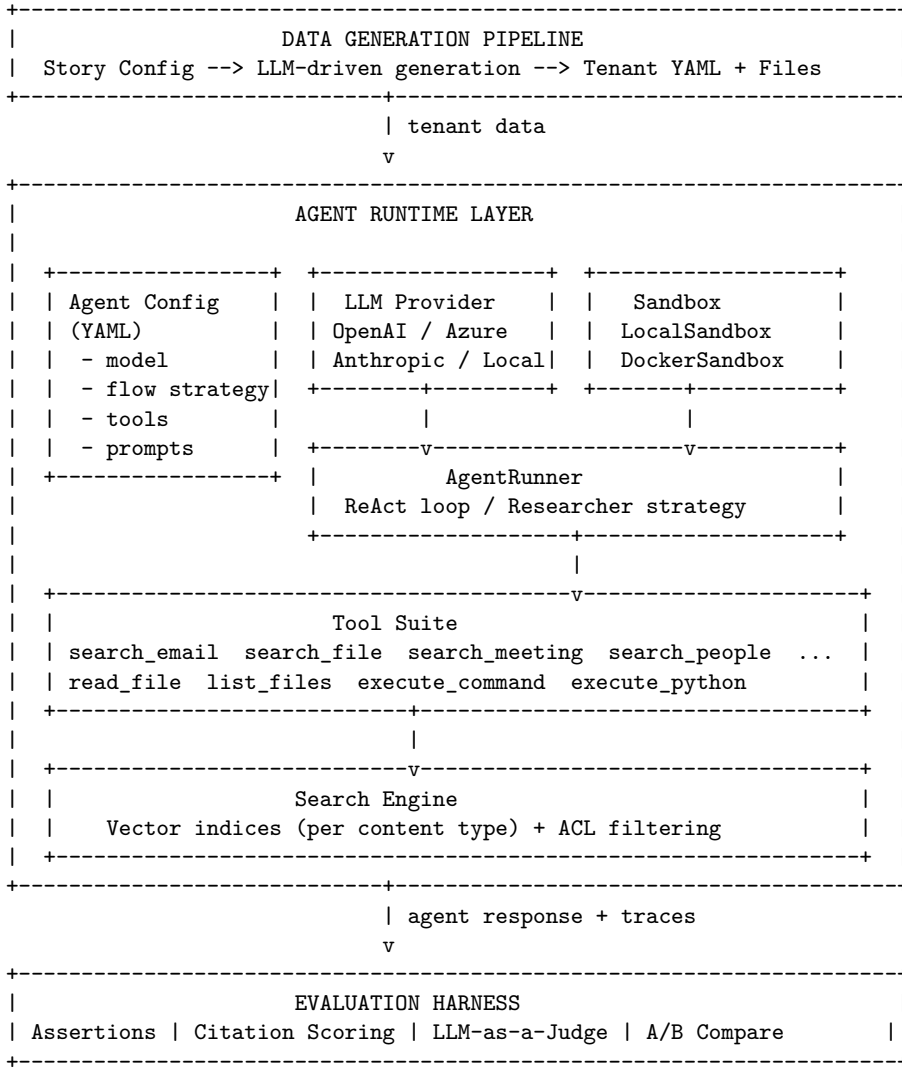


Figure 1: EABench system architecture. The data generation pipeline produces tenant corpora consumed by the agent runtime. The agent uses LLM providers, tools, and a search engine to answer queries inside a sandbox. The evaluation harness scores agent responses along multiple dimensions.

Generator architecture. The data generation pipeline is driven by a `DataGenerator` class that accepts a `StoryConfig` specifying the company name, industry, size, number of users, timeline length, and a list of named *story events* (e.g., “Product Alpha Launch,” “Production Database Outage”). The generator proceeds in five phases:

1. **User generation.** Synthetic employee profiles are created with realistic names, email addresses, departments, job titles, management hierarchies, and skill sets. Users are assigned to organizational groups that determine their communication patterns.
2. **Story arc construction.** For each story event, the generator synthesizes a brief scenario narrative that will inform all subsequent content generation. This ensures that emails, meetings, and files reference the same shared context.
3. **Daily content generation.** For each day in the specified timeline, the generator prompts the LLM to produce a batch of emails, meeting records, instant messages, and file documents consistent with the day’s story state. To maintain coherence across days, the generator maintains a rolling summary of past events.
4. **Cross-referencing.** The generator explicitly instructs the LLM to create cross-references: emails should cite decisions from meetings, files should document outcomes of email discussions, and chats should reference both.
5. **Evaluation query generation.** Given a completed tenant, a separate pipeline generates evaluation queries grounded in the tenant’s content, with LLM-written assertion descriptions that a Judge can later verify.

Example data-generator configuration. Listing 1

shows a representative `StoryConfig`. The `key_events` field provides the narrative arc that drives cross-document coherence: every email, meeting, and file generated for the tenant will reference one or more of these events. The generator produces fully interconnected corpora—emails discussing launch readiness point to the same meeting transcripts in which the go/no-go decision was made, and incident post-mortems in the file store cite the chat threads in which the outage was first flagged.

Listing 1: Example `StoryConfig` for a software-startup tenant.

```
# Data-generator configuration (StoryConfig)
company_name: TechNova Inc.
industry: Software-as-a-Service
company_size: small
num_users: 5
duration_days: 3
key_events:
  - "Product Alpha Launch: TechNova releases its flagship SaaS platform to enterprise customers after months of testing"
  - "Production Outage: An untested code push crashes the live API; on-call engineers execute an emergency rollback"
description: >
  A fast-growing SaaS startup building enterprise productivity tools for mid-market companies, now undergoing a critical product-launch phase.
```

Strengths over prior approaches. Unlike BEIR [Thakur et al., 2021] or other document retrieval benchmarks, EABench data spans *multiple modalities* (emails, meetings, files, chats, channels) and requires *cross-modal reasoning*. Unlike curated enterprise benchmarks such as WorkArena, EABench data can be generated on demand for any domain and scale, enabling evaluation under distribution shift.

3.2 Agent Configuration

EABench adopts a *configuration-as-code* model in which every aspect of an agent is specified in a human-readable YAML file. This enables rigorous ablation studies without code changes and allows practitioners to version-control agent designs alongside benchmark data.

AgentConfig schema. The top-level `AgentConfig` Pydantic model includes:

- `model`: Provider type (`openai`, `azure`, `anthropic`, `local`), model name, and sampling parameters (temperature, top-p, max tokens).
- `embedding`: Provider and model for dense retrieval.
- `system_prompt`: The agent’s instruction prompt, which may include a `{user_profile}` placeholder automatically populated with the active user’s profile at runtime.
- `planning_prompt`: An optional prompt used exclusively during the planning phase of the Researcher strategy.
- `query_analyzer_prompt`: Per-tool LLM prompts for the query analyzer (see §3.3).
- `tools.definitions`: The list of tool names available to this agent, enabling fine-grained capability control.

- `flow.strategy`: Either `react` or `researcher`.
- `flow.max_turns`: Maximum number of ReAct iterations before the agent is forced to return.

Example ReAct agent configuration. Listing 2 shows a condensed YAML for a ReAct agent. The `{user_profile}` placeholder in `system_prompt` is resolved at runtime with the active user’s profile, enabling per-user personalisation without any code changes. The `tools.definitions` list gives practitioners fine-grained control over which tools are exposed to the model.

Listing 2: Condensed ReAct agent configuration (`react_agent.yaml`).

```
id: react-agent-v1
version: "1.0.0"
model:
  provider: azure
  name: gpt-4o
  parameters: {temperature: 0.7}
embedding:
  provider: azure
  model: text-embedding-ada-002
system_prompt: |
  You are an intelligent enterprise assistant.
  ### User Context (Personalisation)
  {user_profile} # resolved at runtime for the active user
  ### Tool Usage Guidelines
  - Search first: use search_email, search_file, search_meeting.
  - Resolve names to User IDs via search_people before other searches.
  - Use execute_python ONLY for data analysis; print all results.

  ### Citations
  - Cite every retrieved fact inline with [~N~] and add a References section at the end of every response.
tools:
  definitions:
    - search_email
    - search_file
    - search_meeting
    - search_chat
    - search_people
    - read_file
    - execute_python
flow:
  strategy: react
  max_turns: 5
```

Example Plan-and-Execute (Researcher) agent configuration. Listing 3 shows the additional `planning_prompt` field used by the Researcher strategy. The planner LLM receives the raw user query and the user profile, and emits a structured step-by-step research plan that is then injected as context for the standard ReAct execution loop. Because planning and execution use separate model invocations, the planner model can be a smaller, faster model (e.g., `gpt-4o-mini`) while the executor uses a more capable model.

Listing 3: Condensed Plan-and-Execute agent configuration (`researcher_agent.yaml`).

```
id: researcher-agent-v1
version: "1.0.0"
model:
  provider: azure
  name: gpt-4o-mini
  parameters: {temperature: 0.0}
planning_prompt: |
  You are a Senior Research Planner.
  Create a step-by-step plan to answer the user’s request.
  User Request: {user_query}
  User Profile: {user_profile}
```

```

Guidelines:
1. Resolve all names to User IDs via search_people first.
2. Break multi-hop questions into dependent steps.
3. Do NOT execute -- output only the numbered plan.
Output format:
1. [Step Name]: description (Tool: <tool_name>)
2. [Step Name]: description (Tool: <tool_name>)
system_prompt: |
... (same enterprise-assistant prompt as ReAct agent) ...
tools:
definitions:
- search_email
- search_file
- search_meeting
- search_people
- read_file
- execute_python
flow:
strategy: researcher
max_turns: 10

```

ReAct execution. Under the `react` strategy, the `AgentRunner` maintains a running conversation history and iterates through LLM call $\rightarrow$ tool execution $\rightarrow$ observation cycles until the LLM produces a response without tool calls, or `max_turns` is reached. The system prompt is injected once at the beginning of the conversation; subsequent turns contain tool results as `tool` role messages.

Researcher (Plan-and-Execute) execution. Under the `researcher` strategy, `AgentRunner` first invokes the LLM with the `planning_prompt` to produce a high-level, step-by-step research plan. The plan is then injected into the conversation as context for the standard ReAct loop. This separation allows the planner to reason globally about the task structure before committing to individual tool invocations. The final response is sanitized to remove internal planning artifacts before being returned to the user.

3.3 Enterprise Tool Suite

EABench provides a comprehensive suite of enterprise tools organized into two categories.

Semantic search tools. Seven search tools cover every content type in the tenant:

- **search_email:** Multi-strategy email search supporting semantic similarity, keyword matching, sender filtering, and recency ranking. A *query analyzer* sub-module uses an LLM to determine the optimal retrieval strategy for each incoming query.
- **search_file:** Vector-based search over indexed file content.
- **search_meeting:** Search over meeting agendas and transcripts.
- **search_chat** / **search_group_chat** / **search_channel:** Search over 1-on-1 chats, group chats, and channel posts respectively.
- **search_people:** User directory search by name, title, skill, or department.
- **search_in_file:** Keyword-based search within a specific file.

All search tools respect user-level access control: results are filtered to documents accessible to the currently active user identity. This enables evaluation of realistic *data visibility* scenarios where agents must answer queries under specific permission contexts.

Execution tools. Three tools enable agentic computation beyond retrieval:

- **read_file / list_files:** Sandboxed file system access.
- **execute_command:** Sandboxed shell command execution with timeout enforcement.
- **execute_python:** Sandboxed Python code execution with stdout/stderr capture and new-file detection.

Tool registration. New tools are registered using a `@registry.register` decorator, which automatically generates a JSON Schema description from the Pydantic `args_schema` class. Dependencies (`sandbox`, `search_engine`, `llm`, `query_analyzer`) are injected automatically by the agent runner based on the tool function’s parameter signature, eliminating the need for manual dependency wiring.

3.4 End-to-End Evaluation and User-Level Experience Simulation

EABench’s evaluation harness is designed to measure *agent quality* across multiple orthogonal dimensions rather than reducing performance to a single score.

Deterministic assertions. Each evaluation case includes a list of assertion descriptions—natural-language statements about what the response should or should not contain. At evaluation time, a judge LLM assesses whether each assertion is satisfied, yielding a discrete pass/fail result per assertion. The *assertion score* is the fraction of passing assertions.

Citation scoring. EABench adopts a structured citation format (`[^N^] Title (Author, Date) [Type: <type>, ID: <id>]`) that enables automated verification. The evaluator extracts all citations from the agent’s response, fetches the corresponding source entities from the tenant, and prompts a judge LLM to score the relevance of each citation to the user’s query. This directly measures *factual grounding*: an agent that fabricates non-existent email IDs or cites an irrelevant meeting receives a low citation score regardless of the plausibility of its narrative.

LLM-as-a-Judge (side-by-side comparison). In addition to absolute scoring, EABench supports *comparative evaluation* of two agent configurations on the same set of queries. A judge LLM receives both responses and determines a winner based on the evaluation criteria, producing a win-rate and a statistical significance estimate via a paired *t*-test. This enables ablation studies (e.g., does the Researcher strategy outperform ReAct on multi-hop queries?) with quantified confidence.

User-level experience simulation. A key feature of EABench is the ability to specify a `user_id` per evaluation

case. When a case is run under a specific user identity, the search engine filters all retrieval results to documents accessible to that user according to the tenant’s group membership rules. This simulates the experience of different personas—an engineer, a manager, a contractor—and enables evaluation of whether agents correctly respect data access boundaries.

Metric aggregation. The final result for each evaluation case is determined by applying minimum pass thresholds to both dimensions: a case *passes* only when `assertion_score` ≥ 0.75 **AND** `citation_score` ≥ 0.70 . Neither threshold alone is sufficient, ensuring that an agent must both satisfy the stated requirements and ground its response in real source entities. Aggregate metrics across a full evaluation set include pass rate, mean assertion score, mean citation score, mean latency, mean token usage, and mean tool call count.

Example evaluator configuration. Listing 4 shows the evaluator (judge) configuration. The `prompts` block defines two LLM prompt templates: `citation_relevance` measures factual grounding by scoring whether each cited source is genuinely relevant to the query, and `assertion_check` verifies whether the agent’s response satisfies each natural-language assertion, producing a per-assertion pass/fail result and a weighted final score.

Listing 4: Condensed evaluator (judge) configuration (default_judge.yaml).

```
name: "Default Judge Prompts"
model:
  provider: azure
  name: gpt-4o
  parameters: {temperature: 0.0}
prompts:
  citation_relevance: |
    You are an expert evaluator. Judge the relevance of
    citations
    used by the agent to answer the user query.
    User Query:      {query}
    Agent Tool Calls: {tool_calls}
    Agent Response:  {response}
    Score 0-1 (1 = perfectly relevant; 0 = hallucinated/
    irrelevant).
    Output YAML:
      score: <float 0.0-1.0>
      explanation: "<string>"
  assertion_check: |
    Verify whether the agent response satisfies each assertion.
    User Query:      {query}
    Agent Response:  {response}
    Assertions:      {assertions}
    For each assertion output passed: true/false and reasoning.
    Compute final_score as the weighted fraction of passed
    assertions.
    Output YAML:
      assertions:
        - {id: 1, passed: <bool>, reasoning: "<string>"}
      final_score: <float 0.0-1.0>
      summary: "<string>"
```

Example evaluation case. Listing 5 shows a representative evaluation case produced by the automated case-generation pipeline for a tenant. Each case binds a natural-language query to a specific user identity (enforcing access control), a set of verifiable assertions, and a list of expected source entities for citation checking.

Listing 5: Auto-generated evaluation case (TechNova tenant, eval_dataset.yaml).

```
- id: case_001
  # Query run under Omar Khan's access context
  query: >
    Show me Priya Patel's email about client app
    performance issues.
  user_id: okhan
  assertions:
    - description: "Returns email titled 'Client Concerns About
      App Performance'"
    - description: "Identifies sender as ppatel"
  entity_list:
    - entity_type: email
      entity_id: email-009
```

Table 1 summarizes how EABench compares with existing benchmarks across key dimensions.

4 Experiments

This section is reserved for experimental results. Forthcoming work will report benchmark scores across multiple LLM backends (GPT-4o, Claude 3.5 Sonnet, Gemini 1.5 Pro, open-source models) and agent configurations (ReAct vs. Researcher, varying tool subsets, varying prompt designs) on a standardized collection of enterprise tenants spanning at least three industry verticals (technology, healthcare, finance). Experiments will investigate:

- How agent strategy (ReAct vs. Researcher) affects performance as a function of query complexity (single-hop vs. multi-hop).
- The impact of LLM backbone on citation accuracy and assertion pass rate.
- The effect of available tool subsets on task success rate.
- Sensitivity of evaluation metrics to judge LLM quality.
- Scaling behavior of the data generator (small: 5 users, 7 days vs. large: 50 users, 90 days).

5 Conclusion and Discussion

We have presented **EABench**, a configurable benchmark framework for evaluating LLM-powered agents in enterprise settings. EABench addresses three gaps in existing evaluation infrastructure: the absence of realistic, at-scale enterprise corpora; the lack of flexible, code-free agent architecture configuration; and the insufficiency of single-metric evaluation for multi-step agentic systems.

Key contributions. EABench’s LLM-driven data generator produces coherent, inter-connected organizational data across six content types (emails, meetings, files, chats, group chats, channels) with controllable scale and narrative structure. Its configuration-as-code agent runtime enables systematic comparison of ReAct and Plan-and-Execute strategies across arbitrary LLM backends without modifying source code. Its multi-dimensional evaluation harness—combining deterministic assertions, citation grounding verification, and LLM-as-a-Judge—provides a more complete picture of agent quality than any single metric alone. User-level experience simulation adds a

Table 1: Comparison of EABench with existing agent benchmarks. ✓ = supported, – = not supported.

Feature	WebArena	AgentBench	ToolBench	τ -bench	WorkArena	EABench
Multi-source enterprise data	–	–	–	–	Partial	✓
Synthetic data generation	–	–	–	–	–	✓
Configurable agent strategies	–	–	–	–	–	✓
User-level access control	–	–	–	–	–	✓
LLM-as-a-Judge evaluation	–	–	–	–	–	✓
Citation grounding verification	–	–	–	–	–	✓
Side-by-side comparison	–	–	–	–	–	✓
Multi-provider LLM support	–	Partial	–	–	–	✓

privacy and access-control dimension that is absent from prior benchmarks.

Limitations. EABench currently supports only English-language content generation and English-language agent evaluation. The data generator relies on a capable LLM (GPT-4-class or better) to produce coherent narratives; lower-quality models may produce less realistic data. The citation format required by the evaluator must be enforced through the system prompt; agents that do not adhere to this format receive a zero citation score even if their responses are factually correct. Finally, the evaluation assertions are LLM-generated and may not capture all dimensions of response quality.

Future work. Several directions are planned:

1. *Docker sandbox integration:* Extend the sandbox backend to support containerized execution for stronger security guarantees when evaluating agents that run arbitrary code.
2. *Adversarial tenant generation:* Create tenants with deliberate ambiguities, contradictory documents, and planted misinformation to evaluate agent robustness.
3. *Multi-agent evaluation:* Extend the runtime to support multi-agent workflows, where specialized sub-agents (e.g., an email-reading agent and a meeting-summarizing agent) collaborate on a shared task.
4. *Multilingual support:* Extend the data generator and evaluation harness to support non-English enterprise environments.
5. *Human evaluation calibration:* Conduct a study comparing LLM-as-a-Judge scores with human annotations to quantify judge reliability.

We believe EABench fills a critical gap between the rapid development of LLM agent capabilities and the infrastructure needed to evaluate them rigorously in the settings where they will actually be deployed. We release all code, data generation scripts, and evaluation tooling under the MIT license to support reproducible research in enterprise AI.

References

W.-L. Chiang, L. Zheng, Y. Sheng, A. N. Angelopoulos, T. Li, D. Li, et al. Chatbot Arena: An open platform for

evaluating LLMs by human preference. *arXiv preprint arXiv:2403.04132*, 2024.

A. Drouin, M. Gasse, M. Caccia, I. H. Laradji, M. Del Verme, T. Marty, et al. WorkArena: How capable are web agents at solving common knowledge work tasks? *arXiv preprint arXiv:2403.07718*, 2024.

E. Glazer, E. Erdil, T. Besiroglu, D. Chicharro, E. Chen, A. Gunning, et al. FrontierMath: A benchmark for evaluating advanced mathematical reasoning in AI. *arXiv preprint arXiv:2411.04872*, 2024.

X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, et al. AgentBench: Evaluating LLMs as agents. *arXiv preprint arXiv:2308.03688*, 2023.

Microsoft. Microsoft 365 Copilot: The AI-powered productivity tool for work. Microsoft Blog, 2023.

A. Mitra, L. Del Corro, S. Mahajan, A. Cotas, C. Simoes, S. Agrawal, et al. AgentInstruct: Toward generative teaching with agentic flows. *arXiv preprint arXiv:2407.03502*, 2024.

Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789*, 2023.

N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *NeurIPS 2021 Datasets and Benchmarks*, 2021.

L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, and E.-P. Lim. Plan-and-Solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *ACL*, 2023.

S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.

S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

- W. Zhao, X. Ren, J. Hessel, C. Cardie, Y. Choi, and Y. Deng. WildChat: 1M ChatGPT interaction logs in the wild. In *ICLR*, 2024.
- L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *NeurIPS*, 2023.
- S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, S. Sridhar, et al. WebArena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.